

SLM and LLM

Ramu Battu and Nitaigour P Mahalik

Department of Industrial Technology, College of Agricultural Science and Technology, California State University Fresno

Abstract

Small language models (SLMs) and large language models (LLMs) are both transformer-based artificial intelligence systems trained to process and generate human language, but they differ in scale, computational demand, and deployment suitability. This poster explains what these two model types are, how they work, and why the distinction matters in practical systems. The discussion is organized around transformer foundations, key model components, model scale, and real-world deployment implications. The literature shows that transformers established the architectural basis of modern language modeling, while scaling research demonstrated that model capability tends to improve as parameters, data, and compute increase in balance. At the same time, work on distillation and efficient model design shows that smaller models can preserve meaningful utility under constrained deployment conditions. The central implication is that model choice should be based on task requirements, infrastructure constraints, privacy needs, and total system cost rather than scale alone. In many real-world environments, SLMs and LLMs should be treated as complementary model classes rather than competing replacements.

INTRODUCTION AND BACKGROUND

Language models are artificial intelligence systems trained to recognize patterns in text and generate or interpret language based on those learned patterns. Modern language models are primarily built on transformer architectures, which improved sequence modeling by using attention mechanisms to process relationships across text more effectively than earlier approaches (Vaswani et al., 2017). Within this broader landscape, SLMs and LLMs represent two important model regimes. Both rely on the same core architectural foundation, but they differ in parameter scale, memory requirements, latency, cost, and operational flexibility. This distinction matters because model size is closely tied to deployment tradeoffs. Larger models often provide broader reasoning and stronger generalization, but they also require more compute, more memory, and more infrastructure. Smaller models may provide less open-domain flexibility, but they are often more practical when speed, privacy, local deployment, and efficiency are critical (Kaplan et al., 2020; Hoffmann et al., 2022; Lu et al., 2024). The central question is therefore not which class is universally better, but which class is technically and economically appropriate for a given application context.

LITERATURE REVIEW

The literature on modern language models can be organized into several connected streams: transformer foundations, scaling and compute research, foundation model framing, and efficiency-oriented model design. Transformer research established attention-based sequence modeling as the dominant architecture for modern natural language processing and made large-scale language training more practical (Vaswani et al., 2017). Scaling-law research then showed that model performance tends to improve systematically as parameters, data, and compute increase, but also that effective capability depends on balancing these resources rather than maximizing only one of them (Kaplan et al., 2020; Hoffmann et al., 2022).A second important stream of literature focuses on the broader role of language models as foundation models, which can be adapted to many downstream tasks and raise both technical opportunities and deployment risks (Bommasani et al., 2021). Within this broader context, large language models have been studied as increasingly capable general-purpose systems with strong cross-domain performance and flexible open-ended generation (Zhao et al., 2023). In parallel, research on efficient model design shows that smaller models can preserve substantial practical value through distillation, curated data, and deployment-aware optimization (Sanh et al., 2019; Lu et al., 2024). More recent SLM literature therefore treats compact models not simply as reduced versions of LLMs, but as a distinct design space optimized for speed, accessibility, and constrained deployment. Taken together, the literature reflects a broader shift from viewing language models only through the lens of scale toward evaluating them in relation to operational fit. This shift supports a more useful framework for understanding SLMs and LLMs: not as a simple hierarchy of quality, but as different model regimes with different strengths, limitations, and deployment purposes (Bommasani et al., 2021; Lu et al., 2024).

METHODOLOGY

This poster uses a structured explanatory literature-review methodology rather than an experimental benchmarking design. Sources were selected and synthesized to explain the foundational concepts behind SLMs and LLMs and to clarify how these model classes differ in architecture, scale, and deployment suitability. The analysis is organized around five main dimensions: transformer foundations key model components model scale and capability efficiency and deployment constraints practical application context This framework treats language-model understanding as a systems-level problem in which model choice is interpreted relative to real-world constraints rather than benchmark performance alone. Instead of asking which class is universally superior, the methodology emphasizes conceptual explanation and fit-for-purpose interpretation using prior literature.

DESIGNS

At a basic level, language models process text as tokens, convert those tokens into embeddings, and use multiple layers with attention mechanisms to refine the relationships across the input. Their learned behavior is stored in parameters, which are adjusted during training. Other important design elements include the context window, fine-tuning, inference, and distillation (Sanh et al., 2019; Zhao et al., 2023).

SLMs contain an approximate range of **100 million to 5 billion parameters**, and **LLMs** contain **above 5 billion parameters**. This distinction is useful because model scale affects memory demand, inference speed, hardware requirements, and practical accessibility. SLMs are generally designed for efficiency, local deployment, and cost control, while LLMs are generally associated with broader reasoning and more flexible general-purpose performance (Lu et al., 2024; Zhao et al., 2023).

Figure 1. Conceptual Trade-Off Between Scale and Efficiency in Language Model Design

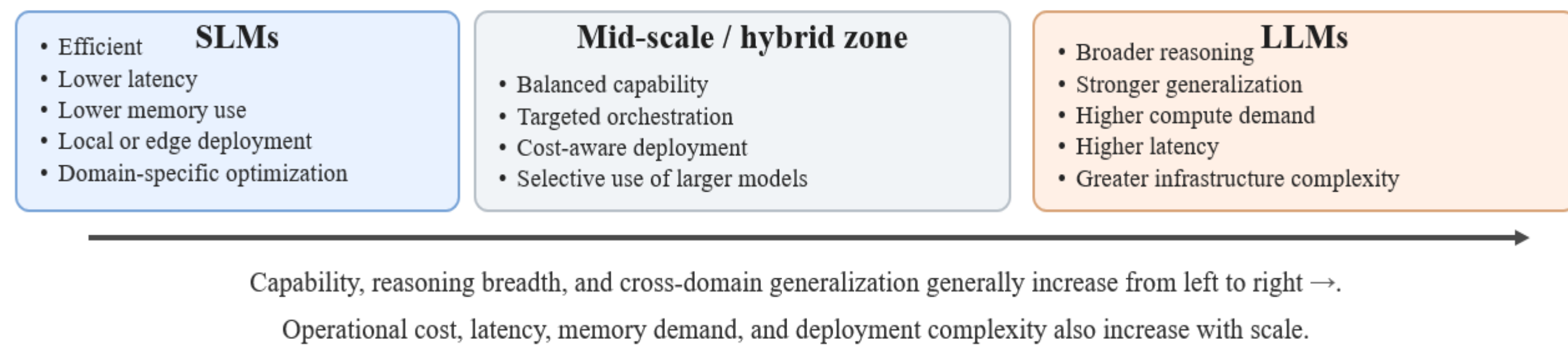


Figure 2. Capabilities of SLMs and LLMs

Category	SLMs	LLMs
Strengths	Low latency, fast inference	Strong generalization across domains
	Low memory footprint	Advanced reasoning + multi-step tasks
	Cost-efficient deployment	Long-context understanding
	Strong performance with curated data	High adaptability
	Edge / local compatibility	Predictable scaling gains
Limitations	Lower parameters: 100M – 5 B	Higher parameters: 5B+
	Lower reasoning capability	High compute + infrastructure cost
	Weak open-domain robustness	Higher latency
	Limited context handling	Difficult edge deployment
Best Use Cases	Sensitive to data quality	Governance + privacy complexity
	Narrow, well-defined tasks	Open-ended tasks
	Real-time systems	Complex reasoning problems
	Edge/on-device deployment	Cross-domain applications
	Privacy-sensitive applications	High-variability inputs

RESULTS

The literature indicates that SLMs and LLMs share the same transformer-based foundations but differ in performance profile and deployment fit. SLM Findings Across the literature, SLMs are associated with: lower latency lower memory demand lower deployment cost stronger suitability for local, mobile, or edge environments better fit for bounded, narrow, or repeatable tasks Their major strength is efficiency. Research on distillation and compact model design suggests that smaller models can preserve useful performance when paired with good data curation and deployment-aware optimization (Sanh et al., 2019; Lu et al., 2024).LLM Findings LLMs are associated with: broader generalization across domains stronger open-ended reasoning better long-context handling more flexible instruction following stronger performance on heterogeneous or ambiguous tasks Their major strength is capability. Scaling and compute-optimal training research indicates that increases in size, data, and compute can produce systematic gains in performance, provided these resources are balanced effectively (Kaplan et al., 2020; Hoffmann et al., 2022).Comparative Interpretation The literature therefore suggests a consistent pattern: SLMs are preferable when efficiency, speed, privacy, and constrained deployment dominate LLMs are preferable when reasoning breadth, flexibility, and general-purpose performance are required This means that model selection should be based on task and system constraints rather than scale alone.

DISCUSSION

he most important implication of the literature is that SLMs and LLMs should not be understood as a simple hierarchy where larger automatically means better. Instead, they should be viewed as different design regimes optimized for different tradeoffs. LLMs remain the stronger option when broad reasoning, cross-domain transfer, and open-ended generation are the priority. However, SLMs remain highly valuable when latency, memory demand, deployment cost, or privacy constraints become decisive. Research on distillation and efficient model design reinforces this point by showing that smaller models can preserve practical value under the right conditions (Sanh et al., 2019; Lu et al., 2024). At the same time, foundation-model research warns that large-scale capability also introduces broader concerns around infrastructure, governance, and operational complexity (Bommasani et al., 2021). As a result, the real contribution of the literature is not the replacement of one class by the other, but a reframing of the model-selection problem around fit between capability requirements and deployment constraints. For enterprise, industrial, and edge systems, this reframing is especially important. In many real-world environments, the most effective architecture may be hybrid. In such cases, LLMs can support broad reasoning, orchestration, and exception handling, while SLMs can support local inference, domain specialization, and cost-controlled routine tasks. This supports the larger conclusion of the poster: model choice should be guided by use case, infrastructure, privacy, and system requirements rather than by scale alone.

REFERENCES

Bommasani, R., et al. (2021). On the Opportunities and Risks of Foundation Models.Hoffmann, J., et al. (2022). Training Compute-Optimal Large Language Models.Kaplan, J., et al. (2020). Scaling Laws for Neural Language Models.Lu, Z., et al. (2024). Small Language Models: Survey, Measurements, and Insights.Sanh, V., et al. (2019). DistilBERT: A Distilled Version of BERT.Vaswani, A., et al. (2017). Attention Is All You Need.Zhao, W. X., et al. (2023). A Survey of Large Language Models.